

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CURSO: DESARROLLO DE APLICACIONES MÓVILES



Ing. Walter Chamorro Palomino



Universidad
Tecnológica
del Perú

Sistema de Evaluación

Promedio Final:

$(10\%)APF1 + (20\%)APF2 + (30\%)APF3 + (40\%)PROY$

DESARROLLO DE APLICACIONES MÓVILES

Unidad 2

Diseño y gestión de datos en aplicaciones móviles

Logro de la Unidad 2:

Semanas 6, 7, 8, 9 y 10

Al finalizar la unidad, el estudiante desarrolla aplicaciones móviles híbridas que gestionan el almacenamiento dinámico de diversas fuentes.



Desarrollo de Aplicaciones Móviles

Introducción a hooks con React native

Semana 6



Universidad
Tecnológica
del Perú

Dudas sobre la sesión anterior



¿Qué trabajamos la sesión anterior?

- Navegación en aplicaciones móviles.
- Configuración e instalación de React Navigation.
- Tipos de navegación: Stack, Tab, Drawer.

Recordando los aprendizajes



¿Cuál es la principal característica de la navegación tipo Stack en React Native?

- A) Permite abrir menús laterales de manera deslizable
- B) Muestra pantallas en forma de pila, donde se puede avanzar y retroceder.
- C) Permite dividir la pantalla en pestañas inferiores
- D) Solo funciona en los.

Recordando los aprendizajes



¿Qué tipo de navegación se utiliza normalmente para mostrar diferentes secciones en la parte inferior de la aplicación?

- A) Stack Navigation
- B) Drawer Navigation
- C) Modal Navigation
- D) Tab Navigation.

Recordando los aprendizajes



¿Cuál de las siguientes afirmaciones es verdadera respecto a React Navigation?

- A) Es una librería oficial incluida en React Native por defecto
- B) Solo funciona con Stack Navigation
- C) No es compatible con Expo
- D) Permite combinar diferentes tipos de navegación en un mismo proyecto.

Recordando los aprendizajes



Si queremos que un usuario pueda acceder rápidamente a las secciones principales de una app desde un menú lateral, ¿qué tipo de navegación deberíamos usar?

- A) Stack Navigation
- B) Tab Navigation
- C) Drawer Navigation.
- D) Switch Navigation.

Conocimientos previos



- ❖ ¿Cómo crees que se puede manejar información que cambia (por ejemplo, un contador o una lista) dentro de una aplicación móvil hecha con React sin usar clases?
- ❖ ¿Qué Hook se utiliza para manejar el ciclo de vida en componentes funcionales?

Utilidad del tema



¿Por qué es importante conocer a los hooks en el diseño de aplicaciones móviles en React Native?

Utilidad:

- *Es importante conocer los Hooks en React Native porque permiten manejar de forma sencilla y eficiente el estado, el ciclo de vida y la lógica reutilizable en componentes funcionales, lo que facilita el diseño de aplicaciones móviles más dinámicas, escalables y fáciles de mantener.*

Logro de la Semana 6

Al finalizar la semana 6, el estudiante comprende y aplica los Hooks en React Native para gestionar el estado y el ciclo de vida en componentes funcionales, diseñando aplicaciones móviles dinámicas y fáciles de mantener.



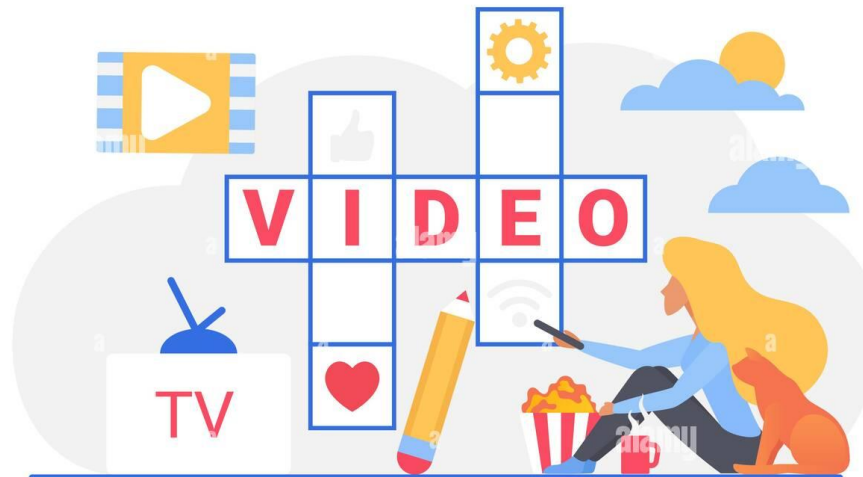
Contenido General

- ❖ Introducción a hooks con React native.
- ❖ Ciclo de vida en componentes funcionales.
- ❖ Integración de Hooks en aplicaciones móviles.



React useEffect en 5 minutos

Observemos el siguiente video y opinemos:



*¿Qué opinas de lo que
has visto en este video?*

Video Youtube:

<https://www.youtube.com/watch?v=7qnRm9F2x50>

*Los participantes responden
de forma oral*

Introducción a Hooks con React Native

- Los Hooks son funciones especiales que permiten usar características de React (estado, ciclo de vida, contexto, etc.) en componentes funcionales, sin necesidad de usar clases.
- Los más usados son:
 1. **useState** → Maneja estados (por ejemplo, un contador, un input de texto).
 2. **useEffect** → Permite ejecutar efectos secundarios (consultas a APIs, temporizadores, listeners).
 3. **useContext** → Consume valores de un contexto global.

Ejemplo sencillo con useState:

```
import React, { useState } from "react";
import { View, Text, Button } from "react-native";

export default function App() {
  const [contador, setContador] = useState(0);

  return (
    <View>
      <Text>Contador: {contador}</Text>
      <Button title="Aumentar" onPress={() => setContador(contador + 1)} />
    </View>
  );
}
```

Ciclo de vida en componentes funcionales

En componentes de clase se usaban métodos como `componentDidMount`, `componentDidUpdate` y `componentWillUnmount`.

En componentes funcionales con Hooks, todo esto se maneja con `useEffect`:

- **Montaje (`componentDidMount`)** → Cuando el componente aparece en pantalla.
- **Actualización (`componentDidUpdate`)** → Cuando cambian las props o el estado.
- **Desmontaje (`componentWillUnmount`)** → Cuando el componente se elimina.

Ejemplo: Mostrar un mensaje cuando el componente se monta.

```
import React, { useEffect } from "react";
import { View, Text } from "react-native";

export default function App() {
  useEffect(() => {
    console.log("✅ El componente se montó");
  }, []);

  return (
    <View>
      <Text>Hola, este es un ejemplo simple de useEffect en JavaScript</Text>
    </View>
  );
}
```


Integración de Hooks en aplicaciones móviles

Los Hooks son fundamentales en apps móviles porque permiten:

- Manejar el estado de forma más simple y legible.
- Usar efectos para conectar con APIs, bases de datos o sensores del dispositivo.
- Compartir lógica entre componentes sin necesidad de herencia (mediante custom hooks).

Ejemplo de integración con una API en un proyecto de restaurante:

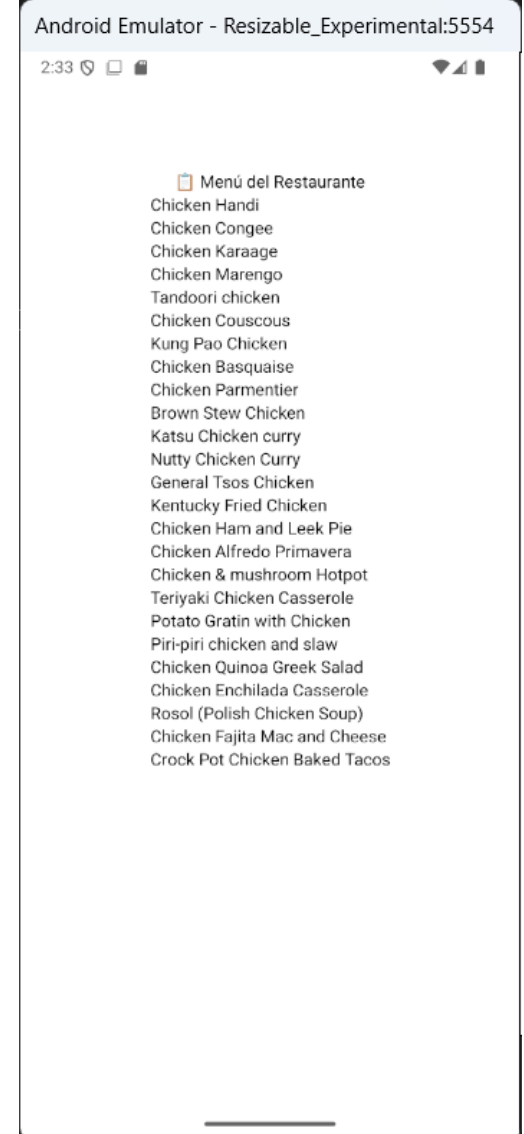
```
import React, { useEffect, useState } from "react";
import { StyleSheet, View, Text, FlatList } from "react-native";

export default function App() {
  const [platos, setPlatos] = useState([]);

  useEffect(() => {
    // Simulación de API
    fetch("https://www.themealdb.com/api/json/v1/1/search.php?s=chicken")
      .then((res) => res.json())
      .then((data) => setPlatos(data.meals))
      .catch((error) => console.log(error));
  }, []);

  return (
    <View style={styles.container}>
      <Text> 🍽️ Menú del Restaurante</Text>
      <FlatList
        data={platos}
        keyExtractor={({ item }) => item.idMeal}
        renderItem={({ item }) => <Text>{item.strMeal}</Text>}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '■ #fff',
    alignItems: 'center',
    justifyContent: 'center',
    margin: 100,
  },
});
```



Preguntas



Desarrollo de Aplicaciones Móviles



Desarrollo de Aplicaciones Móviles

Práctica



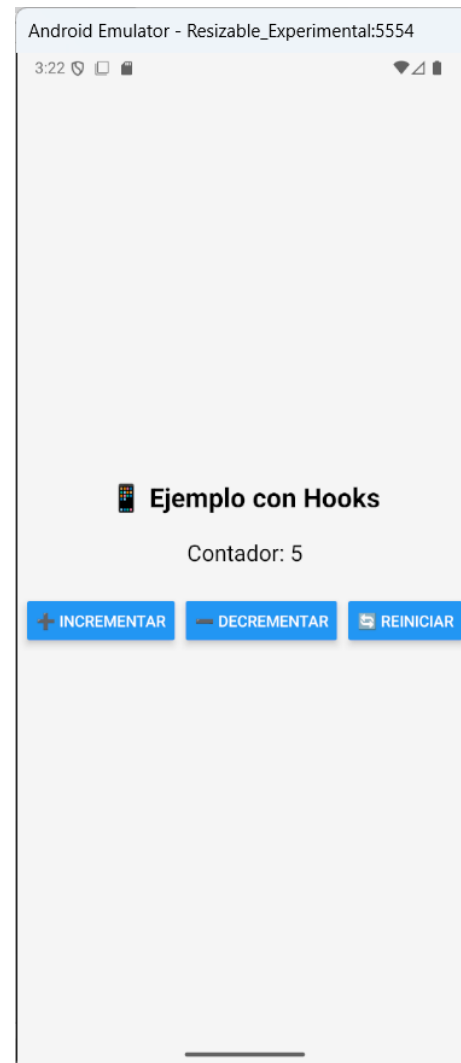
Práctica

Ejercicio 01:

Desarrolle una aplicación móvil en React Native + Hooks en JavaScript, para probar un contador interactivo usando `useState` y `useEffect`.

Explicación:

1. **`useState`** → Crea un estado llamado contador que inicia en 0.
2. **`useEffect`** → Escucha cambios en contador y muestra un mensaje en consola cada vez que cambia.
3. Tres botones permiten **incrementar, decrementar y reiniciar** el contador.



LOG	✓	El contador cambió: 2
LOG	✓	El contador cambió: 3
LOG	✓	El contador cambió: 4
LOG	✓	El contador cambió: 0
LOG	✓	El contador cambió: 1

Práctica

Ejercicio 02:

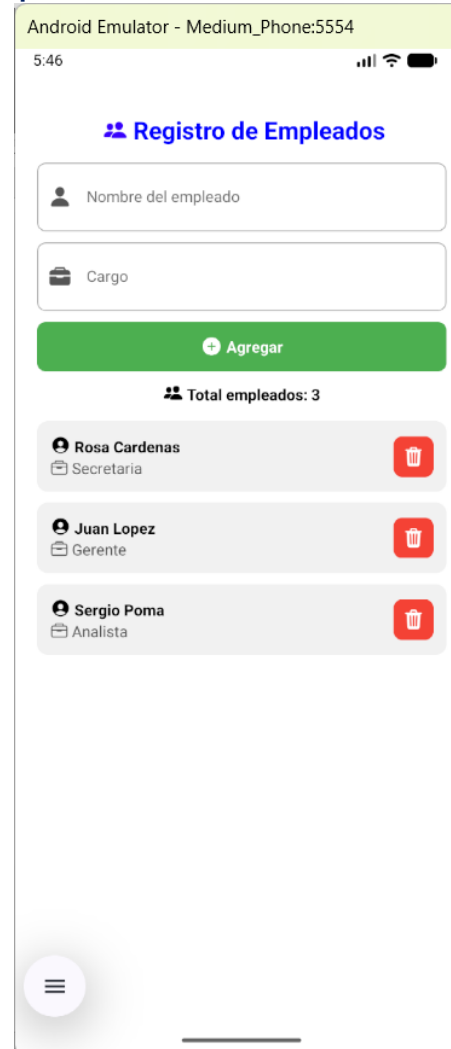
Desarrolle una aplicación móvil en React Native usando useState y useEffect para registrar empleados en una lista, agregando la opción de eliminar un empleado de la lista.

Instalar:

`npx expo install @expo/vector-icons`

Fuente oficial:

<https://icons.expo.fyi/Index>



Preguntas



Desarrollo de Aplicaciones Móviles

Actividad grupal:

En grupos de hasta 8 estudiantes diseñe una aplicación móvil con contenido libre en donde se evidencie los temas tratados en esta semana.



Resumen de la Clase

- Los Hooks simplifican el manejo de estado y ciclo de vida en componentes funcionales.
- El ciclo de vida ahora se controla con `useEffect`.
- Son clave para integrar lógica reactiva en apps móviles, como manejar pedidos, usuarios o integración con sensores del dispositivo.



Preguntas de cierre



- ¿Para qué se usa el hook useState?
- ¿Qué hace el hook useEffect en un componente?
- ¿Por qué los hooks son importantes en el diseño de apps móviles con React Native?

Conclusiones

- Los Hooks simplifican el uso de estado y efectos en componentes funcionales de React Native.
- El ciclo de vida ahora se gestiona con `useEffect`, reemplazando los métodos de clases.
- La integración de Hooks permite apps móviles más dinámicas, conectadas a datos en tiempo real y fáciles de escalar.



Gracias

A close-up photograph of a right hand holding a red pen, positioned to underline the word 'Gracias'. The word is written in a black, cursive script. The background is plain white.